DAY 1

Module 3

Establishing Governance and Writing a Clarified Specification

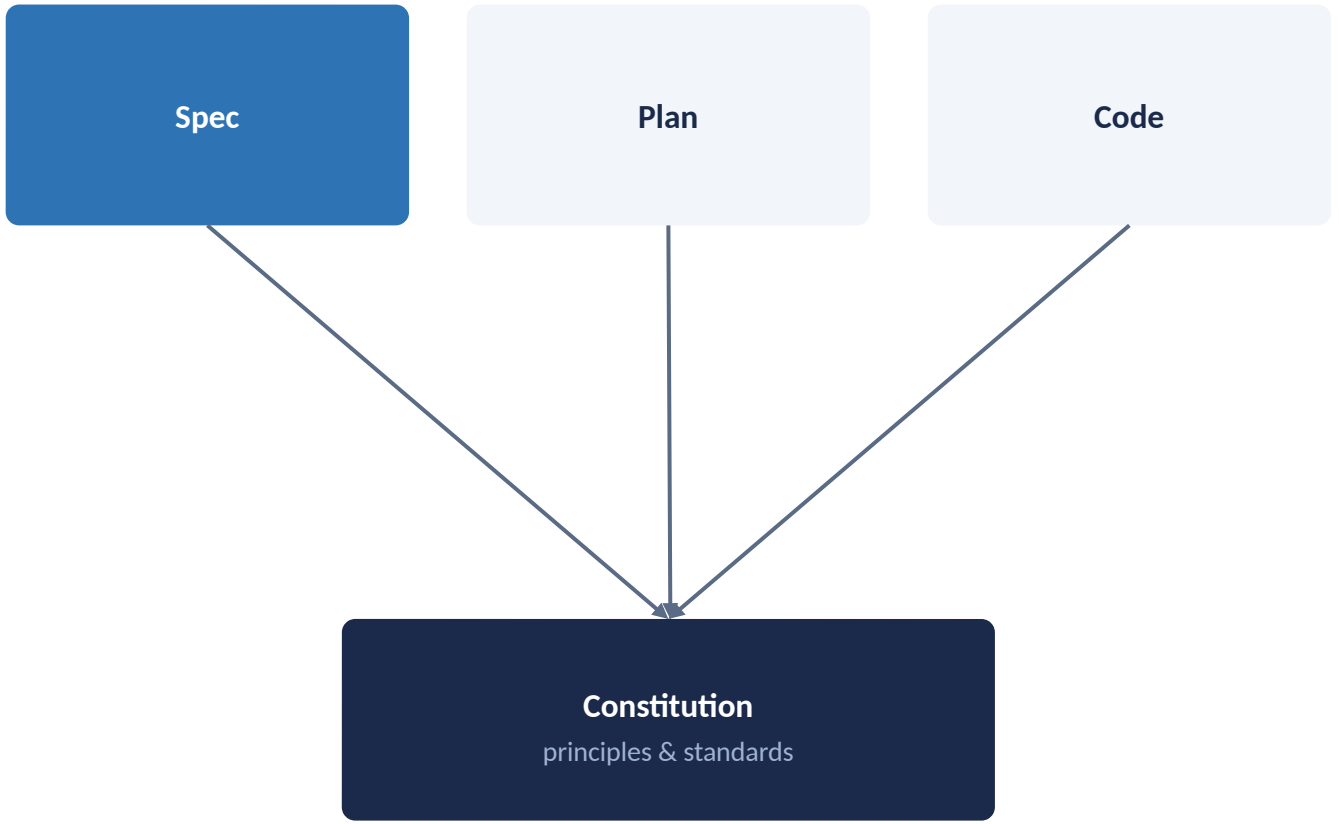
Set the standard everything else gets checked against, then write against it.

What You'll Learn

1. Encode project or per-service governance in a constitution
2. Write a spec from both a feature request and a defect report using EARS-style requirements
3. Run a clarify pass to surface ambiguity before planning
4. Map the SDD lifecycle onto your team's existing SDLC

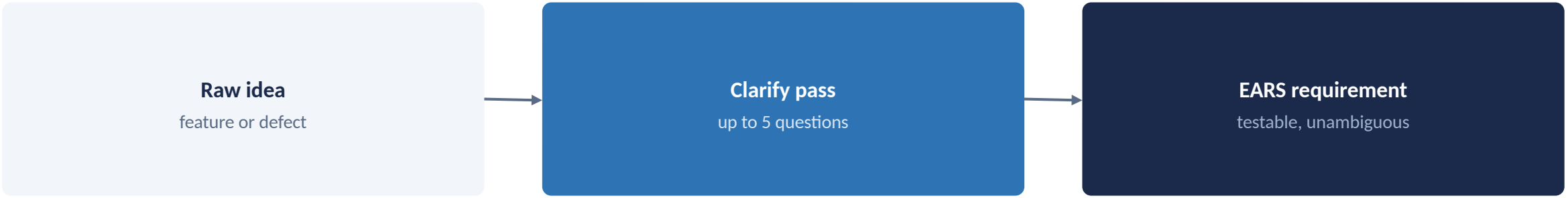
GOVERNANCE FIRST

The Constitution



Every later artifact is checked against the constitution, not against opinion

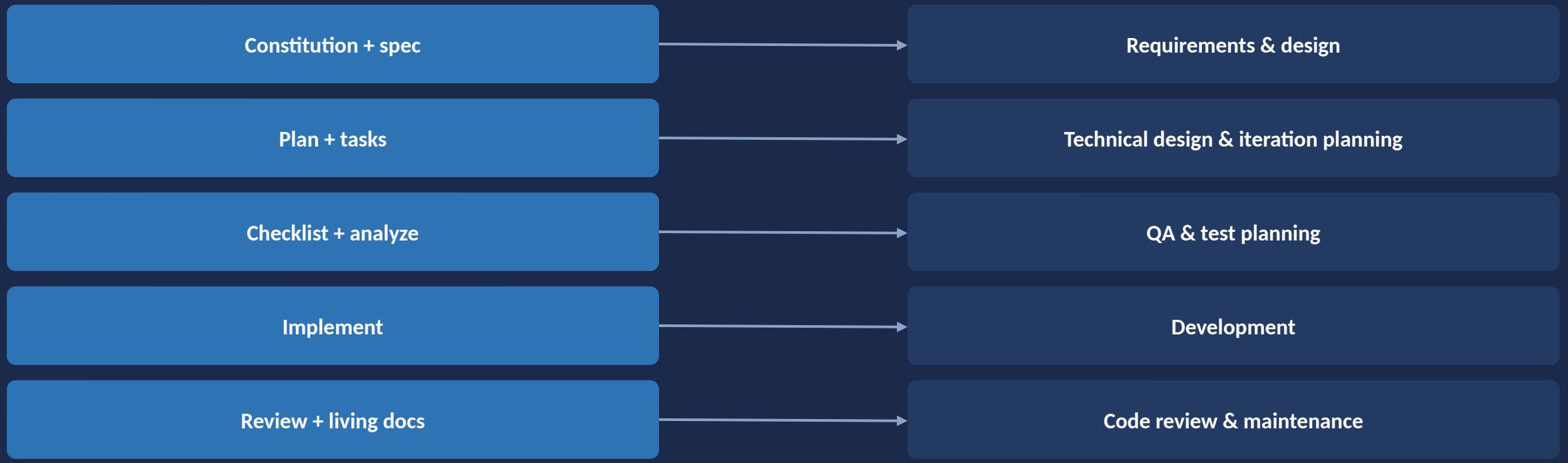
EARS-Style Requirements and Clarify



Ambiguity gets resolved before it reaches a plan, not after

SDLC MAPPING

The SDD lifecycle isn't a separate process: it maps onto the SDLC you already run.



The SDD lifecycle maps onto the SDLC you already run, step for step

APPLYING THE CONCEPTS

GitHub Spec Kit

SPEC KIT

Constitution, Specify, Clarify

`/speckit.constitution`	Creates or updates the constitution and keeps dependent templates in sync, so a change to a principle automatically propagates instead of quietly going stale in one place.
`/speckit.specify`	Creates the feature spec from natural language, focused on what and why, not tech stack, which keeps the spec readable by non-engineers and stable even if the implementation approach later changes.
`/speckit.clarify`	Asks up to five targeted questions about underspecified areas and encodes your answers back into spec.md. Run it as many times as needed, since one pass often surfaces a follow-up question the first round didn't anticipate.

APPLYING THE CONCEPTS

OpenSpec

OPENSPEC

Config Files, Deltas, and Explore

Governance gap: No dedicated governance command: project conventions live in openspec/config.yaml rather than a generated document, so governance is configuration you maintain directly instead of an artifact a command produces for you.

`/opsx:propose`

Generates proposal.md plus delta specs/ marking ADDED, MODIFIED, REMOVED requirements, each with Given/When/Then scenarios: OpenSpec's equivalent of testable acceptance criteria, playing the same role EARS statements play in Spec Kit.

Clarify gap: No single dedicated clarify command: ambiguity gets sharpened beforehand with /opsx:explore, or reconciled afterward with /opsx:update, so the clarify step is distributed across the workflow rather than being one named command.

Module 3 at a Glance

	GitHub Spec Kit	OpenSpec
Governance	/speckit.constitution (dedicated command)	openspec/config.yaml (no generated document)
Spec writing	/speckit.specify	/opsx:propose (delta specs, Given/When/Then)
Clarify	/speckit.clarify (dedicated command)	/opsx:explore before, /opsx:update after

Key Takeaways

- Governance has to exist before code is judged against it, otherwise every review starts from a different, unstated standard.
- EARS keeps requirements testable regardless of which framework carries them, since the format itself, not the tool, is what makes a requirement checkable.
- Clarify early: it's cheap now and expensive once a plan is built on top of ambiguity that nobody caught in time.